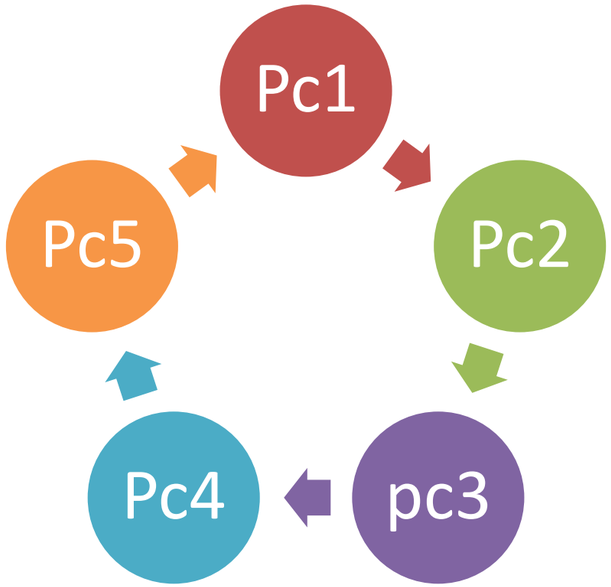
Architecture

- Le système de base de données Cassandra est ***un système orienté colonne***
- Origine: Facebook, Open source: 2088
- Soutenu par ***la fondation Apache***
- ***Utilisaté par:***
  - ***Appel : 70K nodes (100PB)***
  - ***Netflix: 2,5 K***
- **Architecture *en anneaux***
- Favorise la disponibilité (**Read Write Any Where**)

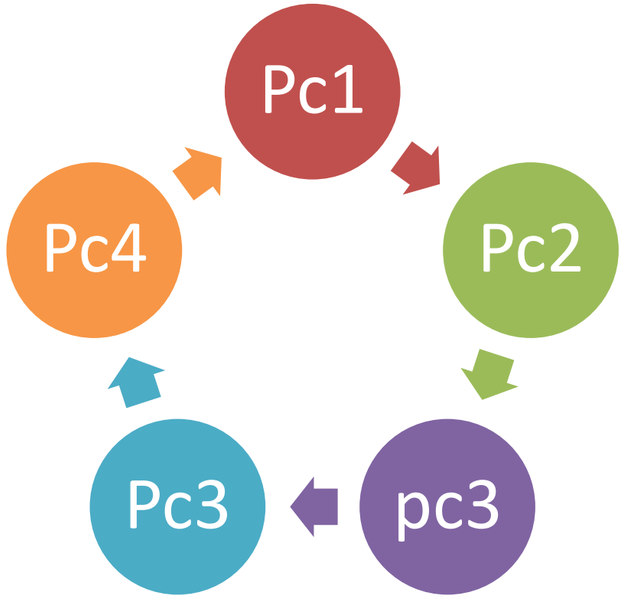
  →n'importe quelle machine **peut accepter** les requêtes des utilisateurs

  → distribution de charge très puissante



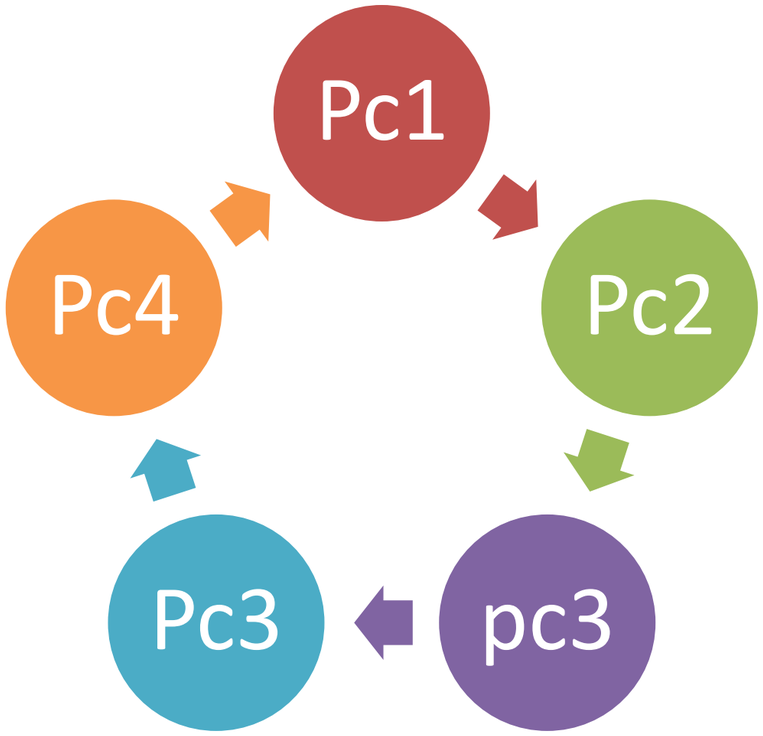
Architecture

- **Cassandra favorise la disponibilité a cause de l'utilisation de l'architecture en anneau**
  - ***Architecture 100 % distribuée***
    - Resistance à la panne (Fault tolerance)
    - Hautement scalable
    - Gros volume
  - Toutes les machines ont les même responsabilités
  - Toutes les machines ont une visibilité sur ce qui trouve sur les autres machines
  - Division équitable des tâches



Architecture

- **Bases de données**
  - Distribuées
  - A haute performance
  - Extrêmement évolutive
  - Tolérance aux fautes
  - Non relationnelle



- **Peut être utilisée à la fois comme:**
  - Bases de données temps réel
  - Bases de données pour une lecture intensive pour les systèmes décisions

Architecture

- **Systèmes distribués, P2P**
- Composé de plusieurs nœuds identiques
  - Dans Cassandra, tous les nœuds d'un cluster sont identiques :
    - **Pas de notion de NameNode ou nœud maître**
  - Cela signifie qu'aucun nœud n'est central ou spécial, supprimant ainsi tout « single point of failure »

- Les données sont partitionnées par défaut à travers les différents nœuds du cluster

- Les données sont répliquées pour assurer une tolérance aux fautes maximale
  - **L'utilisateur contrôle le nombre de répliques qu'il désire avoir pour ses données**

Architecture

- **Systèmes distribués, P2P**

- Les données sont répliquées pour assurer une tolérance aux fautes maximale
  - **L'utilisateur contrôle le nombre de répliques qu'il désire avoir pour ses données**

- Lecture et écriture à partir de n'importe quel nœud, indépendamment de l'emplacement des données

- L'utilisation du protocole **Gossip** pour la communication entre les différents nœuds du cluster
  - **Echange de données entre les nœuds chaque second**

Consistance

- la consistance est un principe fondamental qui détermine à **quel point les données lues ou écrites sont synchronisées entre les différents nœuds du cluster.**
- Contrairement aux bases traditionnelles, Cassandra ne garantit pas de façon stricte la cohérence immédiate, mais permet à l'utilisateur ***de choisir le niveau de consistance adapté à ses besoins.***

Consistance ajustable (tunable consistency)

- Cassandra innove avec la notion de **« consistance ajustable »** : lors d'une lecture ou d'une écriture, l'utilisateur peut spécifier le niveau de consistance requis pour chaque opération.

- Les principaux niveaux de consistance sont :
  - **ONE**
  - **QUORUM :**
  - **ALL :**
- Le choix du niveau de consistance permet de privilégier soit la rapidité, soit la cohérence, selon les besoins métier ou techniques.

Fonctionnement lors des écritures et lectures

- **Lors d'une écriture**,
  - Cassandra écrit la donnée sur plusieurs nœuds selon le facteur de réplication.
  - L'opération est validée quand le nombre de nœuds défini par le niveau de consistance a confirmé la réception de la donnée.

- **Lors d'une lecture,**
  - le nœud coordinateur consulte le nombre de répliques correspondant au niveau de consistance sélectionné et retourne la valeur la plus récente basée sur le timestamp stocké, ce qui permet de gérer les éventuels conflis

Accès universel (Read/Write Anywhere)

- Dans Cassandra, un utilisateur peut se connecter à n'importe quel nœud du cluster, que ce soit dans un datacenter local ou distant, pour lire ou écrire une donnée.
- Cela est rendu possible car chaque nœud joue un rôle de coordinateur, capable de relayer la requête aux nœuds responsables de la partition concernée
-
- **Partitionnement et réplication automatisés**
  - Les données sont automatiquement réparties (partitionnées) à travers tous les nœuds de l'architecture, selon un algorithme de hashage cohérent.
  - Chaque morceau de donnée (« partition ») est répliqué sur plusieurs nœuds, garantissant redondance et disponibilité même en cas de défaillance, selon le facteur de réplication choisi.

Accès universel (Read/Write Anywhere)

Read and Write Anywhere

- L’utilisation peut se connecter à n’importe quel nœud, dans n’importe quel datacenter, et lire/écrire les données qu’il veut
- Les données sont automatiquement partitionnées et répliquées à travers le cluster
- Principe d’ecriture: deux modes de stockage
  - Partie disque
  - Partie en mémoire

Principe d’écriture : mémoire et disque

- Lorsqu’une écriture est initiée, elle suit deux étapes clés :
  - **En mémoire :** La donnée est d’abord stockée dans une structure dite « memtable » (mémoire), ce qui assure rapidité et accumulation des modifications.
  - **Sur disque :** Simultanément, l’opération est enregistrée dans le « commit log » (journal de transaction sur disque) pour garantir la durabilité. Périodiquement, quand la memtable atteint un seuil, elle est vidée (flush) sur disque dans des fichiers appelés SSTables, qui sont des fichiers immuables, triés pour optimiser les lectures ultérieures.
- Cette double gestion assure que les écritures sont rapides et résilientes aux pannes, tout en facilitant la lecture distribuée.

Consistance

- **Ecriture**

  À l'arrivé d'une requête:

  – Données écrites d'abord **sur disque** dans **un commit log** pour la durabilité

  – Ensuite, écriture en mémoire dans **une MemTable**

  – Une fois **la MemTable** pleine, les données sont sauvegardées dans le disque, dans une **SSTable (Stored String Table)**

    - **Commit log contient les requêtes**
    - **SSTable contient les données**

  – Même si les transactions relationnelles (**commits et rollbacks**) ne sont pas supportées, les écritures sont atomiques au niveau des colonnes

    - Soit toutes les colonnes sont modifiées, soit aucune ne l'est

- Cassandra est la base de données NoSQL la plus rapide en écriture

Cohérence éventuelle et réparation

- Par défaut, Cassandra garantit une consistance éventuelle, c'est-à-dire que toutes les répliques finiront par être synchronisées, même si ce n'est pas instantané.

- Des mécanismes de réparation sont utilisés périodiquement pour s'assurer que toutes les copies d'une donnée convergent vers la version la plus récente.

Consistance

- **Consistance:** à quel point est-ce qu'une donnée est à jour et synchronisée sur toutes ses répliques?
- Extension du concept de consistance éventuelle à une consistance ajustable
- Choix possible entre une consistance forte ou éventuelle selon les besoins
- Ce choix est fait par opération, ce n'est pas une stratégie globale pour la base de données (ex, pour changer la stratégie de lecture en quorum)
  - **SELECT** * **FROM** users **USING** CONSISTENCY QUORUM WHERE state='TX'
- Consistance gérée à travers plusieurs data center

Stratégie d'écriture

- **Niveau de consistance:** Combien de répliques doivent être écrites avec succès avant de retourner un acquittement au client?

  - **Any :** une écriture doit réussir sur n'importe quel nœud, au moins un
  - **One (défaut dans CQL):** une écriture doit réussir sur le commit log, et la memtable d'au moins une réplique
  - **Quorum:** une écriture doit réussir sur un certain pourcentage de répliques

    (pourcentage= (facteur de réplication / 2) +1
  - **Local-Quorum:** une écriture doit réussir sur un certain pourcentage de nœuds répliques sur le même datacenter que le nœuds coordinateur
  - **Each-Quorum:** une écriture doit réussir sur un certain pourcentage de nœuds répliques sur tous les datacenters
  - **All:** une écriture doit réussir sur tous les nœuds répliques d'une colonne
    - Offre la plus haute consistance, mais la plus basse distribuées

Stratégie de lecture

- **Read Repair**

A l'arrivée d'une requête

  - **Chercher les données dur les MemTables si elles sont introuvables**
  - **Chercher sur les SSTables**

  – Cassandra assure que les données fréquemment lues soient consistantes
  – À la lecture d'une donnée, le nœud coordinateur compare toutes ses répliques en arrière plan.
  – Si ces données ne sont pas consistantes, envoie une demande de lecture aux nœuds réplicas pour mettre à jour leur données et affiche la donnée la plus récente
  – Read Repair peut être configuré par famille de colonnes et active le mode par défaut

Notions de bases

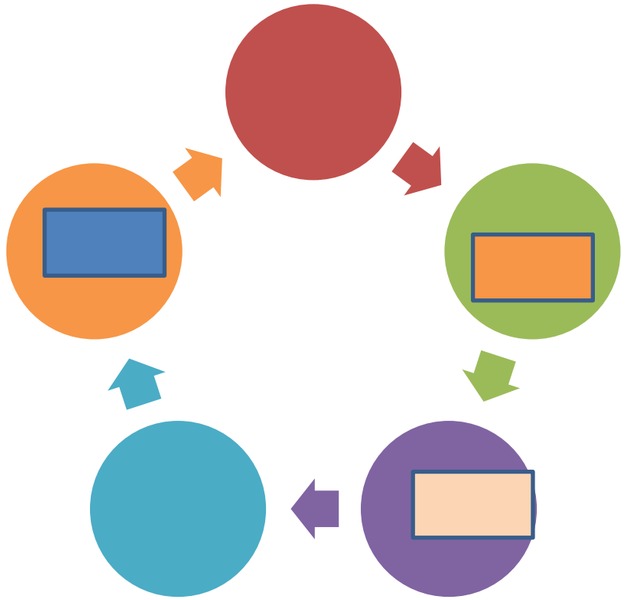
- **Cluster:** ensemble de serveur lié par leurs configuration/réseau, répartie sur un ou pluesieurs DC
- **Datacenter:** groupe de replication, découpage physique ou logique
- **Node/peer:** serveur membre d'un cluster
- **Keyspace:** équivalent à une BD, regroupement de 1 ou plusieurs tables
- **Table:** table composé de lignes et de colonnes
- **Users:** comptes avec des accès
- **Commitlog:** écriture sur disque des requêtes W
- **memTables:** écriture en mémoire
- **Sstables:** datafile ou les données sont écrite sur disque

Notions de bases

- **Coordinateur:** celui qui reçoit la requête en premier,détermine qui peut traiter la requête.
- **Réplica:** copie de donnée distribuées, stratégie de réplication par **keyspace**,
  - Plus on as de replicat plus la latence est importante.
- **Consistance:** combien de réplica doivent être écrite pour considérer la requête comme abouté

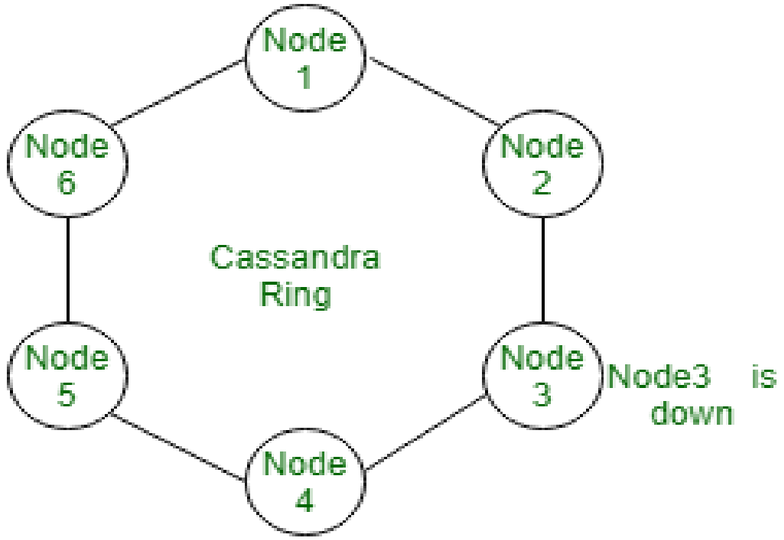
Notions de bases

- **<u>Principe du protocole de Gossip:</u>** principe de découverte de nouds par le cluster et état des nœuds
  - *Gossip est le système de messagerie que les nœuds Cassandra, les nœuds virtuels, utilisaient pour rendre leurs données cohérentes les unes avec les autres, et est utilisé pour appliquer le facteur de réplication dans un cluster.*
- **Exemple:** supposons que dans un cluster Cassandra, chaque nœud contient une certaine partition de chaque table de la base de données.
  - Si une machine fait des modification sur ces données , elle informe ces nœuds adjacents
  - Donc, chaque machine ne peut communiquer l'information qu'avec des nœuds adjacents.



Notions de bases

- **Principe du protocole de Gossip**
  - Dans un cluster composé de 6 nœuds. Supposons que le nœud numéro 3 est en panne
    - Comment Cassandra se comportera dans de telles situations?
    - **le protocole Gossip** dans ce cas est utilisé afin de permettre aux nœuds l'échange périodique des informations d'état sur eux-mêmes et sur d'autres nœuds qu'ils connaissent.



Concept of Gossip Protocol

https://www.geeksforgeeks.org/gossip-protocol-in-cassandra/

Stratégies de partitionnement

Stratégie de Partitionnement 1/4

- Partitionnement facile des données à travers les différents nœuds participants du cluster
- Chaque nœud est responsable d’une partie de la base de données
- Les données sont insères par l’utilisateur dans une famille de colonnes
- Elles sont ensuite placées sur un nœuds selon sa clefs de colonne

Stratégie de Partitionnement 2/4

- **<u>Partitionnement ordonnés</u>**
  - Sauvegarde les clefs de famille de colonnes par ordre à travers les nœuds d'un cluster
  - Peut provoquer des problèmes surtout pour la répartition des charges (des nœuds avec des données plus volumineuse que d'autres)
    - Choisir une colonne parmi les colonnes
      - **Machine 1:** donnée de l'année 2021
      - **Machine 2:** données 2018-2019-2020
      - **Machine 3:** données 2017

Stratégie de Partitionnement 3/4

- **Partitionnement aléatoire**
  - Par défaut, recommandée
  - Données partitionnée le plus équitablement possible à travers les différents nœuds
  - Utilisation de MD5 pour le hachage de chaque clef de famille colonne
    - **Choisir une colonne, en générale l'identifiant → appliquer une fonction de hachage sur cet identifiant → intervalle par machine selon le résultat de hachage**
    - **MD5** est une fonction de hachage cryptographique qui permet d'obtenir l'empreinte numérique d'un fichier

Stratégie de Partitionnement 4/4

- **<u>Partitionnement aléatoire</u>**
  - Par défaut, recommandée
  - Données partitionnéeS le plus équitablement possible à travers les différents nœuds
  - **Utilisation de MD5** pour le hachage de chaque clef de famille colonne

- **<u>Partitionnement ordonnés</u>**
  - Sauvegarde les clefs de famille de colonnes par ordre à travers les nœuds d'un cluster
  - Peut provoquer des problèmes surtout pour la répartition des charges (des nœuds avec des données plus volumineuse que d'autres)

- **Stratégie spécifiée dans un fichier de configuration cassandra.yaml**
- **Si la stratégie d'une base est modifiée, il faut recharger toutes les données**

Technique de réplication

Stratégie de réplication 1/6

- Pour assurer la tolérance aux fautes, il est possible de créer une ou plusieurs copie(s) de chaque colonne à travers les nœuds participants
- L’utilisateur spécifié **le facteur de réplication désiré** à la création du keyspace
- Les données sont insérées par l’utilisateur dans une famille de colonne
- La colonne est répliquée dans les nœuds du cluster selon le facteur de réplication

Stratégie de réplication 2/6

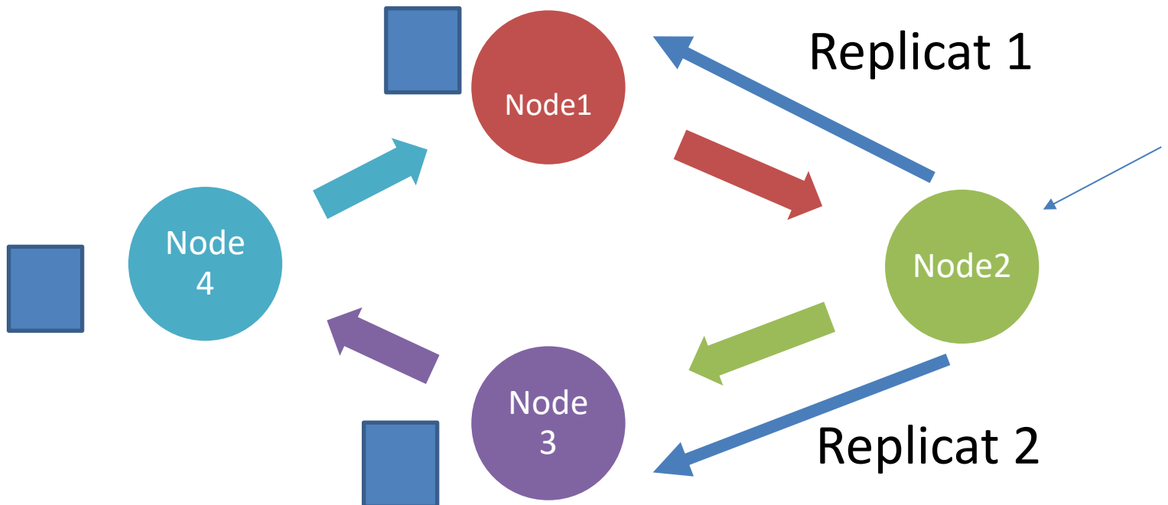
- **<u>Stratégie simple</u>**
  - Dans cette stratégie, elle permet de définir un seul entier **RF (replication_factor).**
  - **Replication Factor (RF) :** Il détermine le nombre de nœuds qui doivent contenir une copie de chaque ligne.

    - **Exemple:** si **replication_factor** est égal à 2, alors deux nœuds différents doivent stocker une copie de chaque ligne.

  - La colonne originalle est placée sur un nœud déterminé par le partitionner
  - La réplique est placé sur le nœuds suivants de l'anneau
  - Pas de considération pour l'emplacement dans un data center ou dans une base (approprié pour les déploiement sur un seul datacenter)

Stratégie de réplication 3/6

- **<u>Stratégie simple</u>**
  - **Avantage**
    - **Simplicité:** rechercher la machine
    - **Rapidité de propagation des répliques des données**
      - Comme chaque donnée, est répliqué dans les voisins dans ce cas d'une modification elle sera propagée des données rapidement → éviter les conflits
  - **Inconvénient** l'emplacement géographique n'est pas pris en considération →gestion des fautes

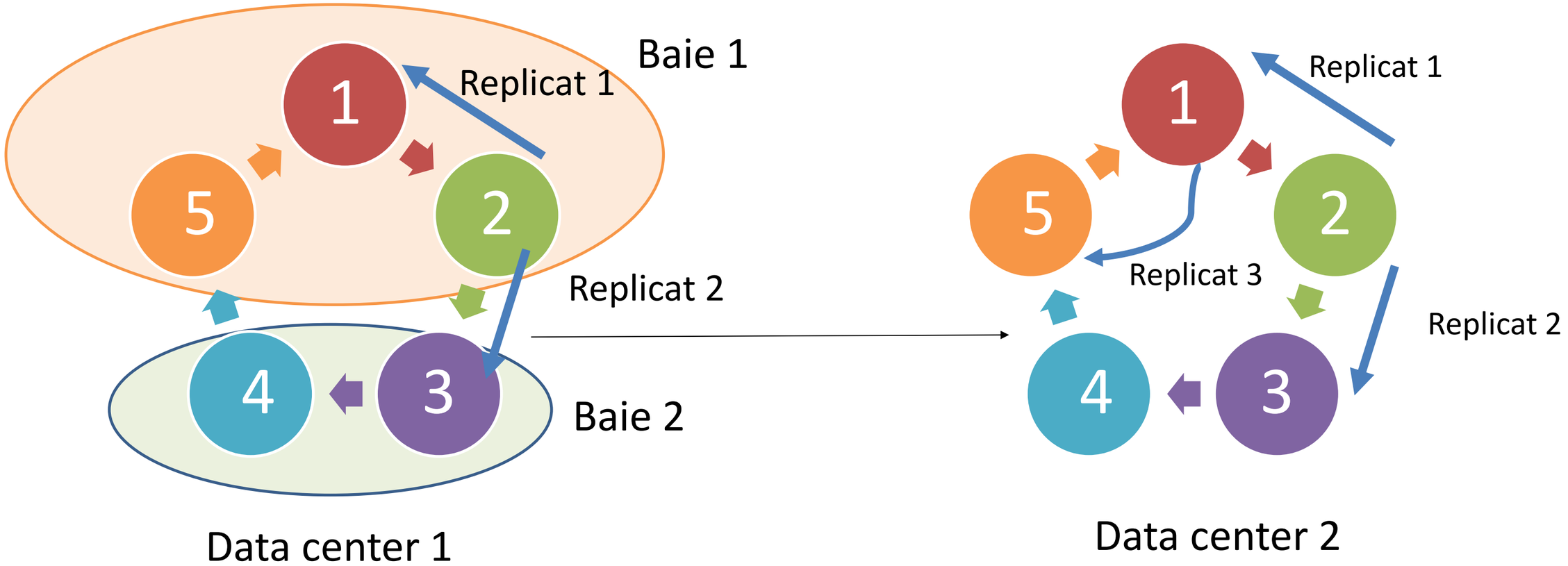


Stratégie de réplication 4/6

- **<u>Stratégie par topologie du réseau</u>**
  - Permet de spécifier un facteur de réplication pour chaque data center du cluster. Même si le cluster n'utilise qu'un seul data center.
  - Plus sophistiquée
  - Plus de contrôle sur l'emplacement des répliques de colonnes
  - Parcourt le cluster à la recherche d'un nœud dans une baie différente. Si introuvable, stocker la colonne dans un nœud de la même baie
  - Un groupe de réplication est spécifié, pour distinguer entre plusieurs datacentres

Stratégie de réplication 5/6

- **<u>Stratégie par topologie du réseau</u>**
  - Avantage
    - faciliter l'ajout ultérieur de nouveaux data center physiques ou virtuels au cluster.



Stratégie de réplication 6/6

- Mécanisme de réplication est basé sur l'utilisation de **SNITCH** qui **vise à trouver au mieux la réponse aux requêtes via la topologie et la méthode de distribution de données**
  - Définition de la manière dont les nœuds sont groupés dans un réseau
  - Répartition des nœuds entre baies et datacenters
  - Plusieurs types de **SNITCH**
    - **Simple SNITCH:** utilise la stratégie simple (n'indique pas ou se trouve chaque machine)
    - **Rack-inferring SNITCH:** détermine la topologie du réseau en analysant les adresses IP
      1. **Second octet de l'adresse IP:** Data Center
      2. **Troisième octet de l'adresse IP:** Baie
    - **Property File SNITCH** se base sur une description de l'utilisation pour déterminer l'emplacement des nœuds (fichier : **cassandra-topology properties**)
    - **Gossiping property File SNITCH**: identifie les data center, et met à jour automatiquement tous les nœuds en utilisant gossip (distribuer l'info entre les machines)
  - Défini dans le fichier de configuration **cassandra.yaml**